

fid-synth___fidelity-chain-f-k4

An AgentMPI run analysis

Abstract

This is the deepest cell of the surrogate series at $p = 16$: a fifteen-deep left fold whose executor is `function` $\times 16$, so it measures the protocol and the surrogate kernel’s arithmetic, not agent behaviour. Structurally it is exact — `fold_depth: 15` $= p - 1$, 15 rounds, 15 messages, model check passing — and its value to the campaign is that it makes the surrogate’s actual failure mode visible in a way the shallower cells cannot. Retention is 0.6875, and the 20 lost facts were not shed gradually: they went in two events. The stored accumulators show that at `merge:d4` on rank 11 the operator discarded the entire accumulated tail of ranks 12 through 15 — sixteen items in one application — and the output collapsed from 862 tokens to 180; one application later rank 10 discarded its own report. Everything after that survived, so the reported `rank_position_correlation` of -0.7751 describes a single contiguous cut near the start of the chain rather than a gradient of positional unfairness. That is the surrogate’s true granularity: because `json.dumps` emits no newlines and the kernel splits its prompt by lines, the items it draws over are whole inputs, so one unlucky draw erases however many ranks that input stood for. The chain is where this is most consequential, since the accumulator arriving at depth d stands for d ranks. It is also where the second pathology is largest: the kernel nests its inputs verbatim, so output grows from 329 tokens at depth 1 to 117,256 at the root, the fabric moves a 77 kB accumulator on the final edge, and the run bills 241,979 output tokens for a fold over 64 four-token facts.

1 What this run is

property	value
run	fid-synth___fidelity-chain-f-k4
experiment	-
campaign label	-
declared world size	16
ranks appearing in the log	16
executors	<code>function</code> $\times 16$
events	95
wall time	0.63 s
job outcome	completed

2 Headline measurements

metric	value	meaning
achieved parallelism	0.10×	total busy time over wall time
parallel efficiency	0.7%	achieved parallelism per rank
idle fraction	99.3%	rank-seconds paid for and unused
peak ranks busy	1	of 16 in the world
serial fraction of active time	100.0%	time with exactly one rank busy
load imbalance	8.00×	slowest rank’s busy time over the mean
coordination share	23.8%	rank-seconds blocked in collectives, of those available
coordination in flight	98.1%	wall clock with any rank inside a collective
rank reattachments	0	fresh agent processes that took over a rank role
agent calls	15	invocations that reached a model
agent latency p50 / p95	0.00 / 0.03 s	per invocation
messages	15	15 eager, 0 rendezvous
tokens sent	125,073	0 deferred under rendezvous
tokens in / out	128,827 / 241,979	prompt and completion
cost	\$4.0162	0.0166 \$/kTok out

3 What the analysis notices

- **[note]** Occupancy is not measurable for this run: its executors (function) emit no broker claim events, so busy time, achieved parallelism, and the idle fraction are artefacts of the instrumentation rather than properties of the run.
- **[warning]** The cost model’s α and β here are a median fallback, not a fit: the slope was positive but the intercept was negative ($\alpha = -0.00$ s), so the fitted line passes below the origin. That is physically impossible and statistically unremarkable on a narrow token range, and the guard rejects it rather than publish a negative fixed cost, on a fit with $R^2 = 0.999$. Any latency prediction from this run’s calibration is a median, not a model.
- **[note]** Load imbalance is 8.00×: the slowest rank was busy far longer than the mean.
- **[ok]** All 1 checkable collective(s) sent exactly the number of messages their closed-form cost expression predicts.
- **[ok]** All 1 checkable collective(s) also matched the predicted round count.

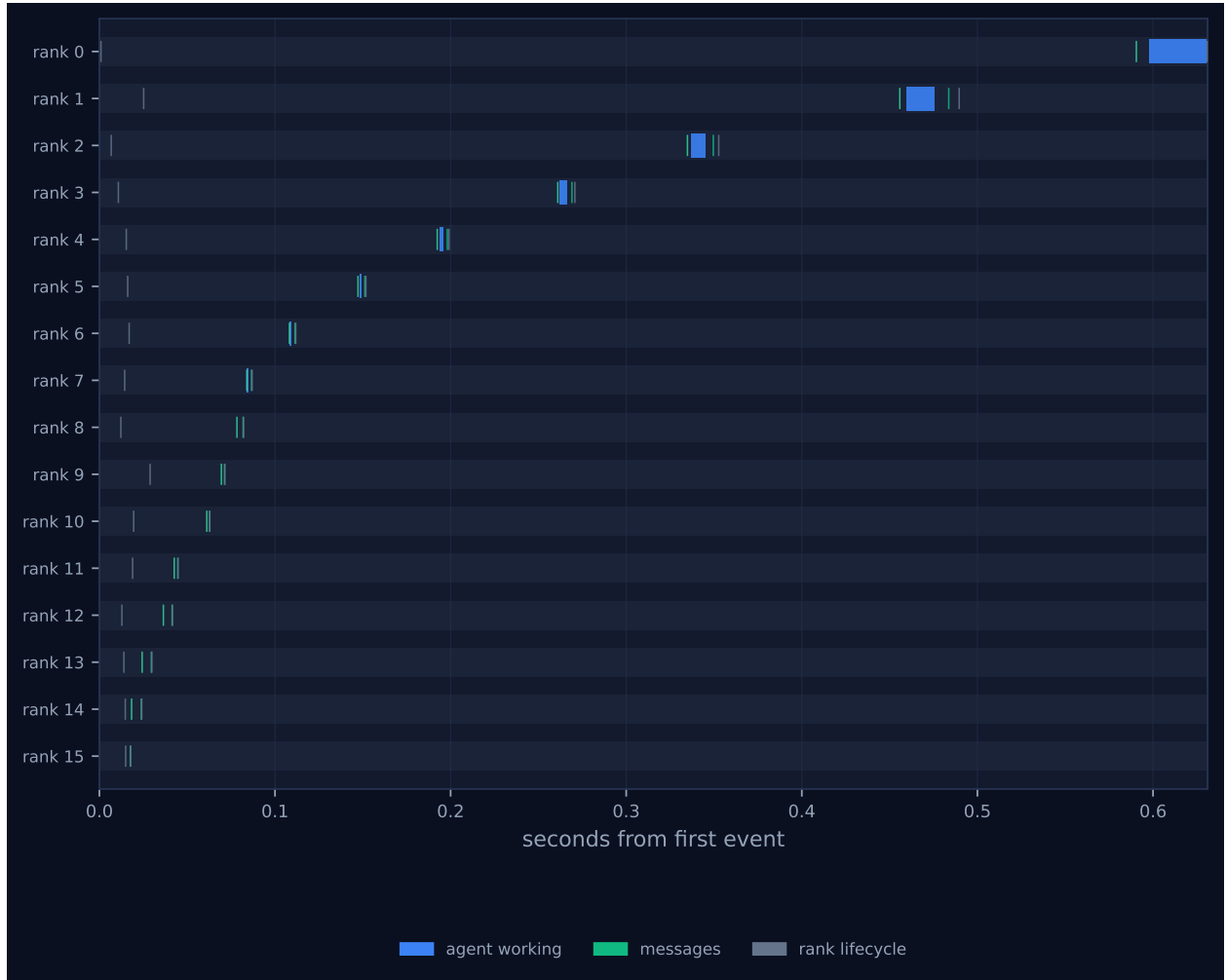


Figure 1: Execution timeline, one lane per rank. Bars are intervals when a rank held a task; ticks are messages; diamonds are window operations, lifecycle transitions, failures, and recovery actions. Drawn in the trace viewer's palette so the same shapes read the same way in both.



Figure 2: Ranks simultaneously busy over time. The dotted line is the declared world size and the dashed line the achieved parallelism; the gap between them is what the run paid for and did not use. Faint vertical lines mark collective invocations.

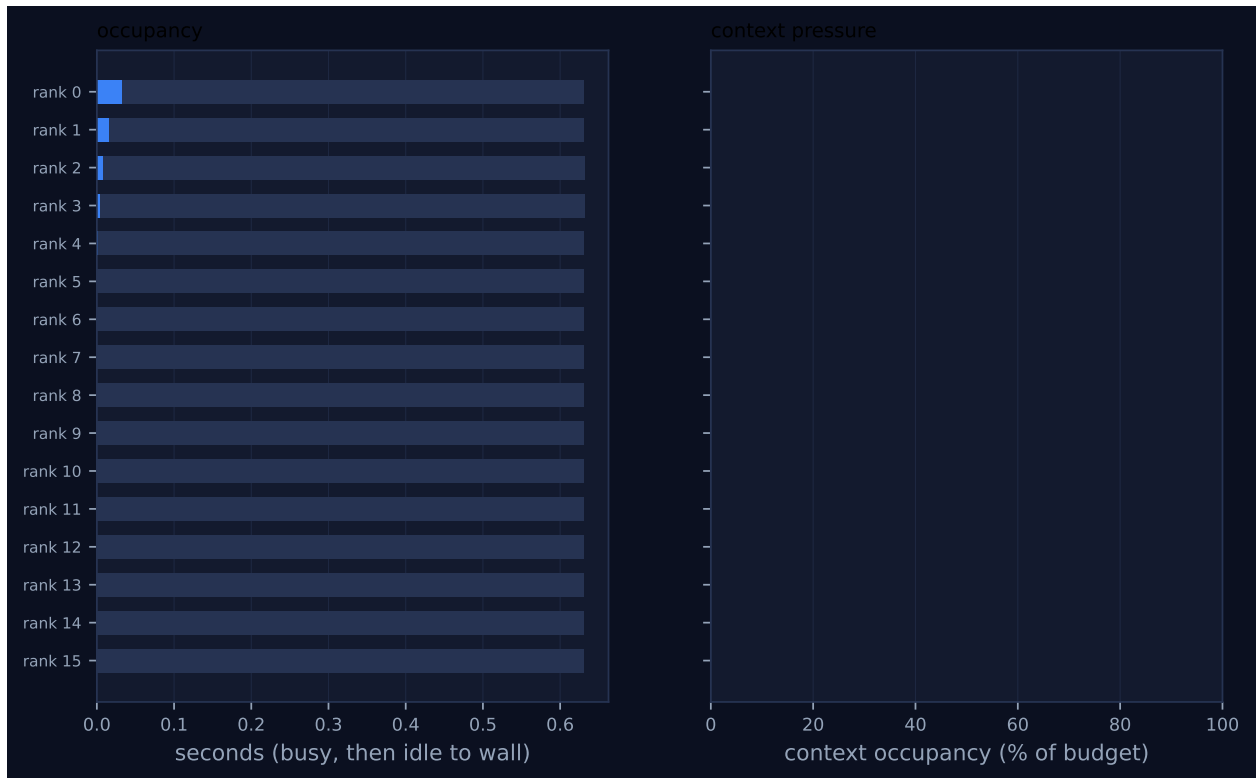


Figure 3: Per-rank occupancy and context pressure. Left: busy time, with the remainder to wall time in grey. Right: context occupancy against budget, amber above half and red above 80%, where admission pressure starts to reject artefacts.

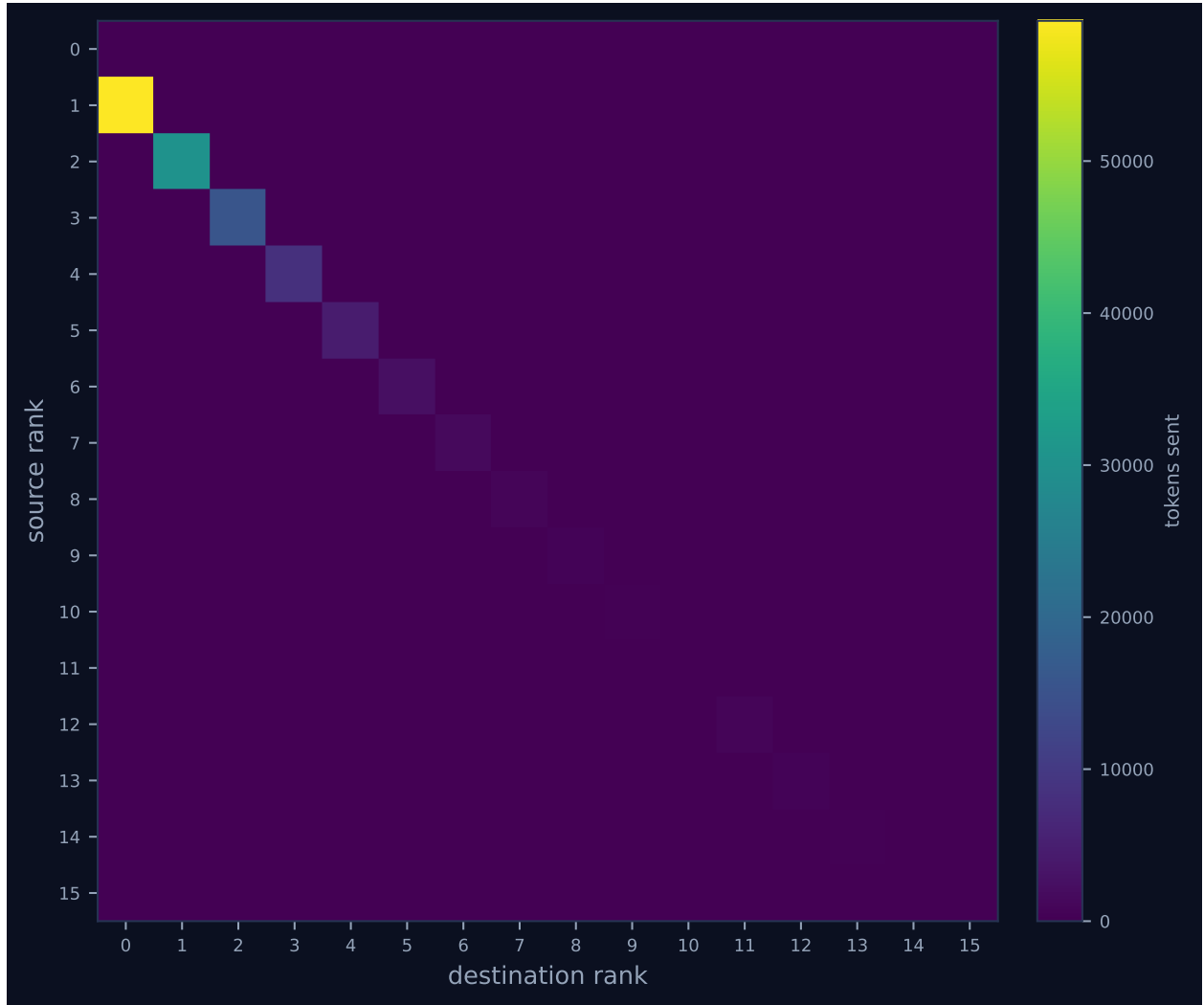


Figure 4: Communication matrix: tokens sent from each rank to each rank. The algorithm’s shape is visible here — a tree concentrates traffic on a root, a ring forms a diagonal band, and an unintended all-to-all fills the plane.

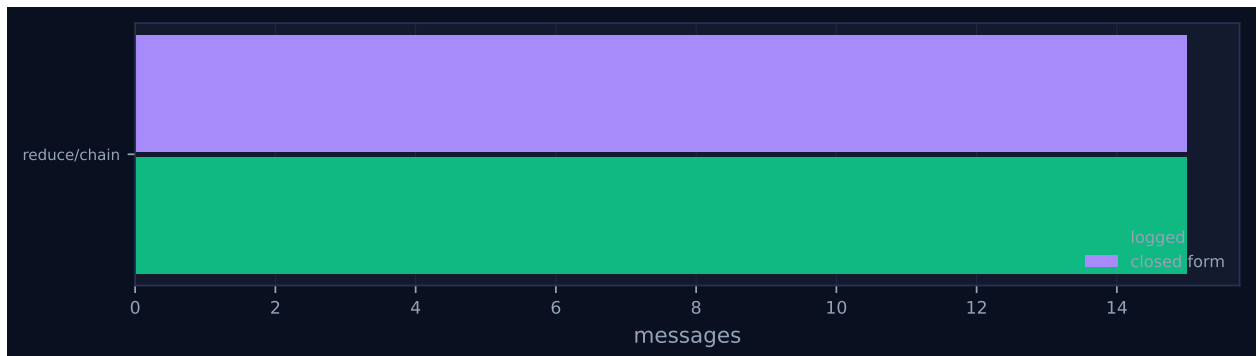


Figure 5: Logged message count against the closed-form prediction, per invocation. A red diamond marks a disagreement between the implementation and its own cost model.

4 Per-rank detail

rank	busy (s)	occ. (%)	tasks	calls	sent	recv	tok. sent	ctx (%)	state
0	0.03	5.2	1	1	0	1	0	0.0	finalized
1	0.02	3.4	1	1	1	1	59,226	0.0	finalized
2	0.01	2.3	1	1	1	1	30,137	0.0	finalized
3	0.00	1.5	1	1	1	1	15,527	0.0	finalized
4	0.00	1.1	1	1	1	1	8,155	0.0	finalized
5	0.00	0.7	1	1	1	1	4,403	0.0	finalized
6	0.00	1.1	1	1	1	1	2,479	0.0	finalized
7	0.00	1.4	1	1	1	1	1,441	0.0	finalized
8	0.00	0.0	0	1	1	1	855	0.0	finalized
9	0.00	0.0	0	1	1	1	496	0.0	finalized
10	0.00	0.0	0	1	1	1	250	0.0	finalized
11	0.00	0.0	0	1	1	1	189	0.0	finalized
12	0.00	0.0	0	1	1	1	877	0.0	finalized
13	0.00	0.0	0	1	1	1	547	0.0	finalized
14	0.00	0.0	0	1	1	1	338	0.0	finalized
15	0.00	0.0	0	0	1	0	153	0.0	finalized

Per-rank behaviour. Occupancy is busy time over the rank’s own lifetime, so a rank that joined late is not penalised for the time before it existed.

5 Collectives, against the cost model

op	algorithm	label	p	seen	rounds	pred.	msgs	pred.	tokens	wall (s)	skew (s)	model
reduce	chain	–	16	16	15	15	15	15	124,867	0.62	0.61	✓

Messages are the count the fabric logged, attributed to each invocation through its internal tag, rather than the count the collective reported — the two are independently produced, and preferring the log is what makes this a check rather than a restatement. A dash in the model column means the comparison is not meaningful: either no closed form exists for that algorithm, or the invocation never completed.

6 Reading the timeline

Figure 1 is a diagonal: rank 15 acts first at $t = 0.018$ s, rank 14 at 0.019, rank 13 at 0.025, and so on down to rank 0 at 0.631s, one bar per lane and never two lanes at once. That is the chain reduction with nothing else in it — $p - 1$ rounds, each round one message and one application, the critical path equal to the whole run — and the peak-ranks-busy figure of 1 of 16 with a serial fraction of 100% is the algorithm’s definition rather than a defect. The $8.00\times$ imbalance is likewise arithmetic, and in any case the generated findings note that occupancy is not measurable for a **function** executor at all, since it emits no broker claim events; achieved parallelism of $0.11\times$ and the 99.4% idle fraction are instrumentation artefacts.

The interesting structure is not spatial but in the widths, which accelerate. Successive per-call latencies are 0.000s for the first eight merges and then 0.001, 0.001, 0.001, 0.002, 0.004, 0.008, 0.016 and 0.033s, so the last application alone is a third of the run. The cause is in the outputs: 329, 532, 862, **180**, 235, 481, 840, 1,426, 2,464, 4,388, 8,140, 15,512, 30,123, 59,211, 117,256 tokens. From depth 5 onwards each application very nearly doubles its predecessor, which is what a kernel that copies both inputs into its output does when one input is the previous accumulator. The fabric pays for it too: the communication matrix, Figure 4, is a clean diagonal band whose weights grow 153, 189, 250, 496, 855, 1,441, 2,479, 4,403, 8,155, 15,527, 30,137, 59,226 tokens up the chain, 125,073 in total, against the binomial tree's 5,416 and the flat fold's 2,133 at the same p . This is the only cell in my set where the reduction's own message traffic is the dominant cost, and it is an artefact of the surrogate rather than of the algorithm.

The one discontinuity in that sequence is the whole story of the run's retention. The fourth merge, on rank 11, took an 1,108-token prompt and returned 180 tokens. Nothing else in the run shrinks.

7 Interpretation

The structure is by construction. `fold_depth: 15` equals $p - 1$, matching the `chain` branch of `algorithms.py`, and this cell is one of the two depth-15 points in the $p = 16$ series — the other being `flat`, which is not a one-shot p -way fold but $p - 1$ sequential binary applications at the root, and therefore has the same measured depth with `rounds: 1` instead of 15. The campaign's four algorithms thus span three depths, 2, 4 and 15, not four. Fifteen messages against a predicted fifteen and fifteen rounds against a predicted fifteen are the model check passing.

What emerged is the granularity of the surrogate's loss, and the chain exhibits it more sharply than any other cell because the quantity at risk grows with depth. Recovering each message payload from `runs/fid-synth/fidelity-chain-f-k4/blobs` and extracting the identifiers gives the fold's history directly. The accumulator leaving rank 12 at depth 3 carries ranks 12, 13, 14, 15 — sixteen real identifiers plus one phantom. The accumulator leaving rank 11 at depth 4 carries rank 11 and the phantom, five identifiers, 189 tokens. One application, four ranks erased. The next accumulator, leaving rank 10, still carries only rank 11 and the phantom, so rank 10 lost its own report as well; and from depth 6 to the root nothing further is lost, the identifier count climbing 9, 13, 17, 21, 25, 29, 32, 36, 40 as each remaining rank is absorbed intact. Five ranks erased, twenty facts, and $64 - 20 = 44$ is the reported `n_retained`.

The mechanism is the same one the binomial cell exposes, and it is a defect in the surrogate rather than a property of reduction. `_surrogate_merge` builds its candidate list by iterating `prompt.splitlines()` and keeping lines matching `FACT_ID`, but each input is interpolated as `json.dumps(...)` and contains no newlines, so a binary merge prompt offers exactly three candidates: one line of the prompt's own instructions — "such as [F-3-2]. Keep every identifier ..." — and the two inputs entire. The stored accumulators confirm this directly, with `findings` of length 2 or 3 throughout and the instruction sentence always first. So the drop the docstring describes as per-item is per-input, and the loss it describes as $(1 - r)^{\text{depth}}$ is instead a lottery in which the stake at depth d is d ranks. The phantom [F-3-2] rides through every accumulator in the run and is scored as a retained item; here it happens to be redundant, because rank 3's real report survived and its per-rank retention is 1.000 either way, but in the binomial sibling the same phantom is the sole reason rank 3 reads 0.25.

Read against the campaign, then, this cell contributes structure and a caution, not a retention

datum. Its 0.6875 ties the k -ary cell's and sits below the flat fold's 0.875 despite sharing the flat fold's depth, which is enough on its own to show that the numbers in this series are not tracking depth. The reason flat did better is not shape but draws: its fifteen prompts all have distinct lengths so its fifteen draws are effectively independent and only two came up short, whereas here one early draw took four ranks at once. Neither observation is evidence about operators. The evidence on the depth hypothesis is in the broker campaigns, where the four algorithms agree exactly with one another — at 1.000 under an advisory budget and at 0.3854 under an enforced one — and where the governing variable is demonstrably the enforced output budget. This run is in the advisory regime itself: at the provenance commit `c5e8e9e` the harness passed the module-level `MERGE_CONTRACT`, which has no `max_tokens` field, so the 900-token budget was a sentence in the prompt and nothing more. That is why a 117,256-token output records `attempt: 1` with `contract_violations: 0`. A contract check that cannot fail is not a check, and the flagged zero should be read that way.

The remaining flagged finding is the calibration warning, and it is expected here for a reason worth naming: the fit was rejected because it produced a negative intercept ($\alpha = -0.0001\text{s}$) on $R^2 = 0.9987$. The relationship really is almost perfectly linear in output size — the surrogate's cost is string concatenation — but the intercept is smaller than the timer's resolution, so the guard fires. That is the guard behaving correctly on a run whose latencies are microseconds, not a defect, and it means the published $\alpha = 0.003\text{s}$ and $\beta = 10^{-6}\text{s/token}$ are medians rather than a model.

8 Threats to this reading

The executor is a surrogate, so no claim here transfers to agents. In particular the runaway token growth, the 125,073 tokens of message traffic and the modelled \$4.0162 are properties of a kernel that concatenates rather than summarises; a real operator asked to merge two reports into at most 900 tokens would not return 117,256, and the broker chain cells at $p = 8$ return 387–695. Anyone comparing wall times or costs across my fifteen runs must partition them by executor first.

The fold reconstruction is a measurement, but an indirect one. The `function` executor writes no rows to `agent_calls`, so this run's fabric holds no prompts and no results; what I read were the fifteen message payloads under `blobs/`, which are the accumulators as sent, and I inferred each application's effect by differencing consecutive payloads along the chain. That is sound for the interior of the chain but says nothing about the root's own final application, whose result was never sent — the claim that ranks 0 through 9 all survived to the end rests on the results file's `per_rank_retention`, not on a blob I read. Similarly, “the drop at depth 4 took the accumulator rather than rank 11's own report” is read off the identifier sets, which is unambiguous, but the reason that particular draw failed is not recoverable without replaying the kernel, and unlike the binomial cell's five identical prompts I did not replay it here.

Two accounting caveats. `tokens_unadmitted: 125,073` equals `tokens_sent`, and every `msg.recv` records `admitted: false`, because the collective's internal receives pass `admit=False`; so context occupancy reads 0% on a rank that received a 59,226-token payload, and this run says nothing about admission pressure even though it is the one run in my set that would obviously have exhausted a real context. That is a genuine gap. And `tokenizer_exact: false`, so all token counts are the repository's approximation.